Model card: PokéVault card recognition

This card follows the structure of Mitchell et al. (2019), "Model Cards for Model Reporting". The system it describes is **not a trained machine-learning model**. It is a fixed pipeline: card outline detection, perceptual-hash matching with hand-set thresholds, and a pretrained third-party OCR model. It is documented in model-card form because its outputs are probabilistic, can be wrong, and are used to make decisions about what is added to a collection. Facts come from the code in this repository and from the evaluation in `eval/page-scan`.

Model details

- **Developer:** Ryley Griffiths, as part of PokéVault.
- **Date:** October 2026.
- **Version:** the code at the tag `element-b-submission` of this repository.
- **Licence:** the code is not licensed for reuse (all rights reserved). Third-party components keep their own licences (below).
- **Contact:** through the GitHub profile.

Type. Three parts, all running in the user's browser (`web/src/scan/` and `shared/cardprint.ts`):

1. **Card finding** (`detect.ts`, `locate.ts`, `warp.ts`).
    - Finds the four corners of the most card-like rectangle in a frame: colour edges, then a Hough transform limited to near-upright lines, then the best combination of four lines with a card's proportions.
    - Then straightens the card with a perspective transform.
    - When a card is in a binder pocket or a sleeve, it also considers other outlines and the margins sleeves leave.
2. **Picture matching** (`shared/cardprint.ts`, `lib/pictures.ts`).
    - Computes a 286-bit perceptual hash of the straightened card. The data card describes how.
    - Finds the nearest of about 21,000 reference fingerprints by Hamming distance, trying the card both ways up.
    - Decides with fixed thresholds (below) whether the closest match is clear.
3. **Text reading** (`ocr.ts`, `parse.ts`, `lib/match.ts`).
    - When the picture can't settle it, for example between reprints with the same artwork, reads the card's name and number with Tesseract OCR.
    - Uses tesseract.js 7.0.0 (Apache-2.0) with its pretrained English LSTM model (`eng`, `4.0.0_best_int`).
    - Ranks catalogue cards by what was read.

Whole binder pages (`pageScan.ts`) apply parts 1 and 2 to each pocket of a photo lined up with an on-screen grid, in two modes:

- **Check:** in a binder in the app, each pocket is compared with the card that belongs there.
- **Read:** in the scanner, each pocket's card is recognised from scratch.

Decision thresholds. These are Hamming distances out of 286 bits. They were set by hand during development using simulated data. None are learned.

Threshold	Value	Used for
Clear match	≤ 70 bits, and the next different card ≥ 20 bits further away	Naming a card from one look
Reprint "twins"	within 40 bits of the best, same name	Offering reprints as choices (the printed number decides)
Settled over several looks	average ≤ 82 bits, gap ≥ 14 bits	Auto-scan of a moving card
Page check: the pocket's card is there	≤ 82 bits, and ≥ 10 bits closer than any other card	Ticking a pocket in a binder
Page read: "probably"	≤ 82 bits, gap ≥ 10 bits	Offering a likely card, unticked

Intended use

- **Primary use:** identifying English Pokémon Trading Card Game cards from a phone camera or photo, so a collector can add them to their collection quickly: one card, a stack one at a time, or a whole binder page.
- **Users:** collectors using PokéVault.
- **A person always confirms.** Nothing is saved without the user's confirmation:
  - A single card is added only after the user picks the match and confirms the details.
  - Page scans tick only clear results, and the user reviews and can untick any pocket before adding.
- **Out of scope:**
  - Judging whether a card is genuine.
  - Grading a card's condition.
  - Non-English cards.
  - Any automated decision without a person's review.
  - Recognising anything other than catalogue cards.

Factors

These factors were expected to affect accuracy:

- **Card factors:** era and layout (vintage and modern frames), reprints that share artwork, mostly flat-coloured cards (some Energy cards), holographic foil.
- **Capture factors:** glare, lighting, blur and camera shake, sleeves and top loaders, binder pocket plastic, page alignment with the guide, background.

The evaluation reports results overall and for cards in penny sleeves. It does not break results down by era, card type, page layout or device. Adding a per-layout and per-era breakdown to `score.mjs` is a recommended next step.

Metrics

The metrics put the cost of a silent mistake first: a wrong card, or a card in an empty pocket, being added without the user noticing. They are:

- **False acceptance:**
  - a wrong card, or an empty pocket, ticked as the pocket's card (check mode)
  - a card ticked as the wrong card, or an empty pocket ticked as a card (read mode)
- **Found rate:** the share of present cards ticked correctly.
- **Not sure:** present cards left for the user to decide. This is a cost in effort, not a mistake.
- **Exact card among the choices:** whether the user can pick the right printing from what is offered.

The thresholds were chosen so that false acceptance is zero on the development data, accepting more "not sure" results in return.

Evaluation data

- **Data:** simulated photos of binder pages (data card, dataset 5).
- **Held-out set:** seed 3, 60 pages, 468 cards. It was generated from the catalogue built on 28 October 2026 and used only for evaluation.
- **Development sets:** seeds 1 and 2, 80 pages, regenerated with the same catalogue. They are reported for comparison.

The simulated photos include sleeves, glare, haze, misalignment, tilt, noise, blur, empty pockets and wrong cards.

Training data

There is no training data, because nothing is learned:

- **Reference fingerprints:** computed from the catalogue's card pictures (data card, dataset 2).
- **Thresholds:** set by hand during development using simulated pages from the same generator, with seeds 1 and 2 and an earlier catalogue build.
- **OCR model:** Tesseract's English model was trained by the Tesseract project, not as part of this one. It is downloaded with tesseract.js and used unchanged.

Quantitative analyses

Held-out set (seed 3, 60 pages); full results are in `eval/page-scan/README.md`:

Measure	Check a binder's page	Read a page of any cards
Present cards	423	468
Found and ticked correctly	393 (92.9%)	402 (85.9%)
Ticked wrongly (false acceptance)	0 of 45 wrong cards	0
Empty pockets ticked (false acceptance)	0 of 42	0 of 42
Left for the user ("not sure", "probably" or not recognised)	30	66 (19 offered as "probably", all right)
Cards in penny sleeves found	116 of 124 (93.5%)	not reported
Exact card among the choices offered	not applicable	435 (92.9%)

The two development sets gave similar results (93.4% found in check mode; 86.3% in read mode; no false acceptances). In read mode, 2 of the 24 "probably" suggestions were wrong; these are not ticked by default.

Not evaluated reproducibly in this repository:

- The single-card and stack scanners.
- Accuracy on real photographs.
- Speed on phones (the evaluation took about 1 second a page per mode on a desktop PC).

Some code comments record figures from earlier development experiments whose scripts are not included (in `shared/cardprint.ts` and `web/src/lib/pictures.ts`, for example). They should be treated as development notes, not results.

Ethical considerations

- **Privacy:**
  - Recognition runs entirely in the browser. Camera frames and photos are not stored or sent to PokéVault's API.
  - Starting the OCR engine downloads its program and language model from the public jsDelivr CDN. That request reveals the user's IP address to the CDN, but sends no images.
- **Mistakes:** a wrong identification leads to a wrong card or value in the collection. This is mitigated by human confirmation and by favouring "not sure" over guessing. No result is used for anything beyond the user's own collection.
- **Intellectual property:** the reference fingerprints are derived from card pictures owned by The Pokémon Company. Only the fingerprints are published, not the pictures.
- **Security:** adversarial robustness has not been assessed. Perceptual hashes can be deliberately collided, so a crafted image could be made to match a different card. In this app, the impact is limited to a user misrecording their own collection.

Caveats and recommendations

- The reported accuracy is from simulated photos built from the same pictures as the reference fingerprints, so it is expected to be optimistic. The next step is a small labelled set of real phone photos of cards and binder pages, with results reported on it.
- Report results by page layout, card era and card type, and measure speed on a phone.
- The single-card scanner needs a reproducible evaluation like the page scanner's.
- Thresholds are global. Cards that are mostly flat colour, and reprints with shared artwork, are the main known sources of "not sure" results.